Recursive / Iterative Retrieval RAG

A practical guide to searching again when the first evidence is incomplete.

1. Start with ordinary RAG

Retrieval-Augmented Generation (RAG) combines document search with a language model. Documents are split into passages, converted into numerical vectors called embeddings, and indexed. A question is embedded too; the retriever selects relevant passages, and the model uses them to write an answer.

A one-shot system retrieves once and then answers. This can miss a crucial rule when the question depends on facts spread across several passages. Similarity to a question does not mean a passage contains everything needed to answer it.

2. Add a feedback loop

Iterative retrieval repeats search when an evidence check finds missing information. Each search targets a specific gap, while useful evidence from earlier attempts is retained. The final answer still addresses the original question.

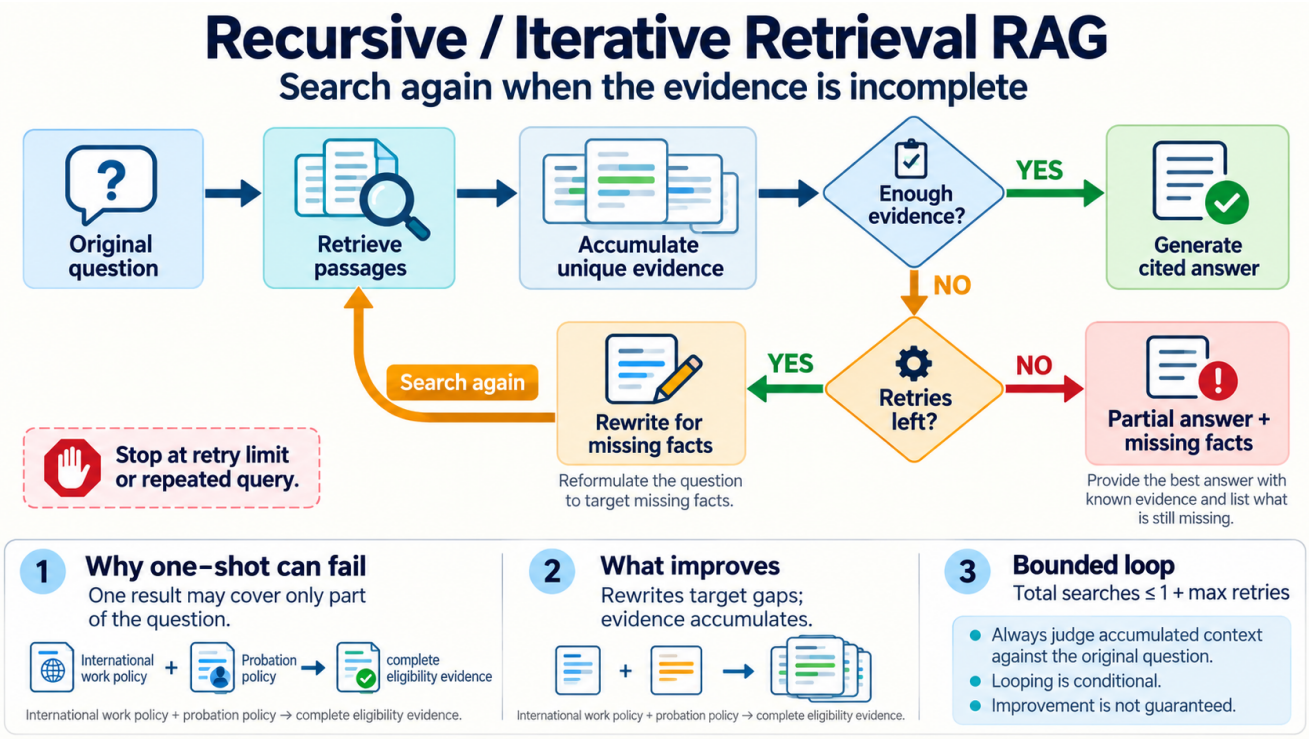


Figure 1. The project flow: retrieve, accumulate, assess, and rewrite if needed. A retry limit or a repeated query stops further searching.

How the retrieval loop works

The implementation in this repository is a bounded iterative workflow.

Step	What happens and why
1. Prepare the corpus	Split documents into chunks with stable IDs. Embed each chunk. This project keeps normalized vectors in memory.
2. Search	Embed the current query, compute cosine similarity, and retrieve the top-k passages. Here, k means passages returned per search.
3. Accumulate	Merge new passages into the evidence pool. Deduplicate by chunk ID so repeated hits do not create duplicate context.
4. Assess	Ask the model whether the accumulated passages explicitly answer every part of the original question. Record missing facts and supporting IDs.
5. Refine or stop	If evidence is sufficient, stop. Otherwise, rewrite the search toward missing facts, unless the retry budget is exhausted or the new query repeats.
6. Generate	Use the accumulated passages to answer with chunk citations. If evidence remains incomplete, identify the unanswered parts.

Iterative versus recursive: related, but different

Iterative retrieval advances through successive attempts in a loop: search, inspect evidence, refine the query, and search again. Its state includes prior queries, gathered passages, and remaining retries.

Recursive retrieval usually means that retrieval invokes further retrieval to resolve a dependency or expand a hierarchy. For example, a question is split into subquestions, each subquestion retrieves evidence, and their results are combined. Another pattern retrieves a summary first, then follows it to underlying document chunks. Terminology varies across systems.

A recursive workflow needs a base case, such as a resolved subquestion, plus a depth or work limit. It may also use an iterative loop within each branch. Repeating a search alone does not imply recursive function calls.

In this project: `Pipeline.run()` in `rag.py` uses a Python `for` loop. It rewrites one query at a time; it does not call itself or build a tree of subquestions. The precise description is **iterative evidence gathering**.

A worked example

Illustrative retrieval sequence using the fictional Atlas handbook. Actual search order depends on the model and embeddings.

Question: Can an Atlas employee who joined 20 days ago work remotely from Lisbon for 20 days, and who must approve it?

Attempt	Evidence and decision
First search	Suppose H001 is retrieved. It says Lisbon is approved, international remote work is limited to 30 days per year, and requests need manager and People Operations approval. But eligibility is governed by a separate probation policy.
Gap check	The location, duration, and approvers are known. The missing fact is whether someone who joined 20 days ago is eligible. Answering now could incorrectly imply permission.
Refined search	An illustrative rewrite is: "Atlas probation eligibility international remote work new employee 90 days". Suppose it retrieves H002: probation lasts 90 days, and employees are ineligible during probation.
Combined answer	No. An employee who joined 20 days ago is still within the 90-day probation period and cannot use temporary international remote work [H002]. After probation, they may apply; Lisbon is approved, the annual limit is 30 days, and manager plus People Operations approval is required [H001].

Bounds and stopping conditions in this repository

With **max_retries = 3**, the pipeline performs at most **4 searches**: the initial search plus three retries. With **top_k = 1**, the evidence pool holds at most four unique passages, or fewer if results overlap. In general, unique context is bounded by **min(corpus size, (1 + max_retries) x top_k)**.

The loop stops on sufficient evidence, an exhausted retry budget, or a repeated query (compared case-insensitively). A provider error ends the run with an error. The original question remains the reference for every assessment, reducing drift from query rewriting.

Benefits, costs, and checks

This approach can recover evidence that the initial search missed, especially for questions joining several policies or facts. Each extra round adds embedding and model calls, latency, and cost. It cannot recover a fact absent from the corpus, and an incorrect sufficiency judgment can stop the loop too early.

Inspect evidence recall, answer support, missing facts, search count, and latency. Compare against a one-shot baseline with a comparable context budget: improvement may come partly from retrieving more passages. Valid citation IDs do not prove that every answer claim follows from the cited text.

Project references

README.md: workflow, retry semantics, and demonstration.
rag.py: Pipeline.retrieve(), Pipeline.run(), Euri.assess(), rewrite(), and generate().
data/handbook.json: fictional H001 and H002 policies.
docs/visuals/iterative-rag-concept.png: source image for Figure 1.